

ASPM IA FIAP

Desafio Pride Security 2026

Application Security Posture Management com Inteligencia Artificial

FIAP - Turma 1TDCPV

Equipe de Desenvolvimento

Paulo Andre Carminati - RM570877

Gustav Quental Scorsi - RM569862

Andre Archanjo dos Santos Torres - RM570458

Luiz Carlos da Paixao dos Santos - RM573009

Victor Henrique de Barros Oliveira - RM570012

Maio de 2026

1. Visao Geral do Projeto

O ASPM IA FIAP e uma aplicacao modular e escalavel desenvolvida para gerenciar a postura de segurancade aplicacoes de forma inteligente, utilizando inteligencia artificial local integrada com GPUs AMD e NVIDIA.

O projeto apresenta duas versoes complementares:

- API Python CLI com integracao de IA local via GPU (NVIDIA CUDA e AMD ROCm) através do LM Studio
- Flask Web como interface remota moderna e responsiva para acesso centralizado

Este documento apresenta a arquitetura completa, modulos, tecnologias utilizadas, fluxos de processamento e guias de execucao para ambas as versoes, destacando o diferencial inovador da integracao com IA local.

2. Objetivos e Escopo

Objetivo Principal

Criar uma plataforma robusta para Application Security Posture Management (ASPM) que integre inteligencia artificial local com aceleracao por GPU, permitindo analises seguras de projetos sem depender de APIs externas ou acesso a internet.

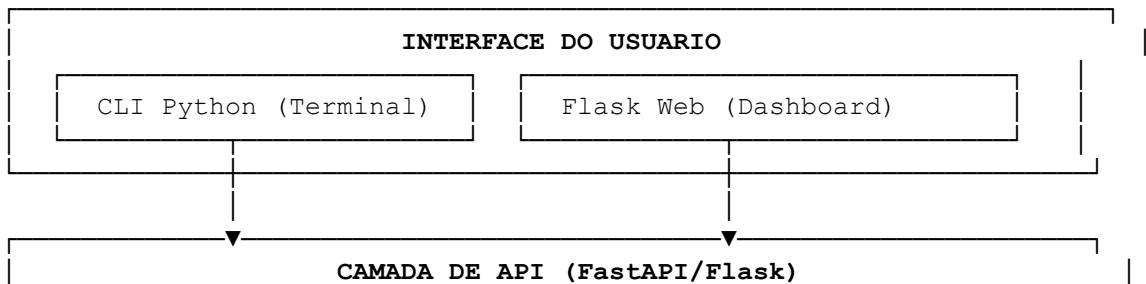
Objetivos Especificos

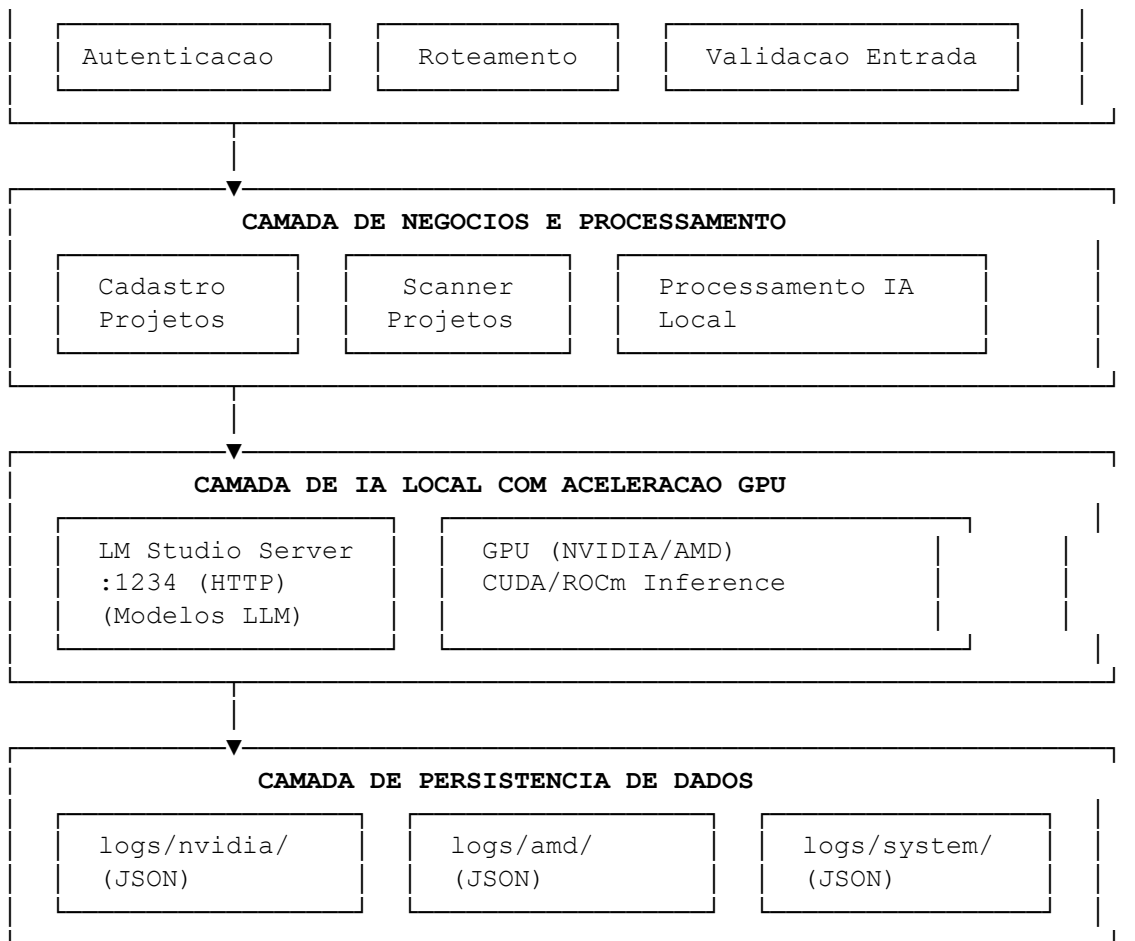
- ✓ Desenvolver arquitetura modular e escalavel que suporte adicao de novos modulos e funcionalidades
- ✓ Integrar modelos LLM locais via LM Studio com suporte nativo para GPUs NVIDIA e AMD
- ✓ Implementar scanner inteligente que detecta vulnerabilidades em projetos automaticamente
- ✓ Providenciar interfaces de usuario (CLI e Web) intuitivas e eficientes
- ✓ Garantir persistencia de dados com relatorios em JSON organizados por tipo de GPU
- ✓ Implementar Dockerizacao para facilitacao do deploy em diferentes ambientes

3. Arquitetura Tecnica

A arquitetura do ASPM IA FIAP foi projetada seguindo principios de modularidade, separacao de responsabilidades e escalabilidade. O sistema funciona em camadas bem definidas:

Diagrama de Arquitetura - Visao Geral





Descricao de Cada Camada

1. Camada de Interface do Usuario

Responsavel por fornecer acesso ao sistema atraves de duas modalidades distintas: CLI (Command Line Interface) para usuarios avancados e Desktop Web para interface moderna e acessivel. A CLI oferece controle total via terminal, enquanto a Flask Web fornece dashboard visual intuitivo.

2. Camada de API

Implementa toda a logica de roteamento, autenticacao e validacao de requisicoes. Garante que apenas requisicoes validas alcancem os modulos de negocio. Utiliza FastAPI na CLI e Flask na Web, ambos fornecendo endpoints REST para comunicacao.

3. Camada de Negocios

Contem a logica principal da aplicacao: cadastro de projetos, execucao de scanners de vulnerabilidades, orquestracao de chamadas para IA local e gerenciamento de logs. Esta camada coordena todas as operacoes do sistema.

4. Camada de IA Local com GPU

Integra-se com LM Studio via HTTP na porta 1234. LM Studio fornece interface para carregar e executar modelos LLM (Large Language Models) com aceleracao por GPU NVIDIA CUDA ou AMD ROCm. Esta camada processa analises complexas de forma rapida e segura localmente.

5. Camada de Persistencia

Armazena todos os resultados em arquivos JSON organizados por tipo de GPU (nvidia/, amd/, system/).
Permite rastreamento completo de todas as analises realizadas e facilita auditoria e reproducao de resultados.

4. Duas Versoes do Projeto

O ASPM IA FIAP foi implementado em duas versoes complementares. Ambas compartilham a mesma logica de negocio e arquitetura, diferenciando-se na forma de acesso e interface.

Criterio	API Python CLI	Flask Web
Interface	Linha de Comando (Terminal)	Web Dashboard (Browser)
Usuarios	Developers e DevOps	Usuarios Finais e Gestores
IA Local GPU	✓ Nativa (NVIDIA/AMD)	✓ Nativa (NVIDIA/AMD)
Performance	Ultra rapido, sem overhead web	Rapido com interface responsiva
Acesso Remoto	Local apenas	✓ Acesso remoto via HTTP

5. API Python CLI - Detalhes Tecnicos

A versao CLI e uma aplicacao Python de alto desempenho otimizada para usuarios avancados que desejam automacao total e controle fino sobre execucoes.

Arquitetura da CLI

```
main.py                (Ponto de entrada da aplicacao)
├── utils/
│   ├── scanner.py      (Modulo de leitura e analise de projetos)
│   ├── llm_client.py   (Integracao com LM Studio HTTP)
│   ├── gpu_detector.py (Deteccao automatica de GPU)
│   └── file_operations.py (Persistencia de dados JSON)
├── models/
│   ├── project.py      (Modelo de dados para projetos)
│   └── analysis.py     (Modelo de dados para analises)
└── logs/              (Diretorio de armazenamento)
```

Modulos Principais

1. Scanner (scanner.py)

Responsavel por ler recursivamente estruturas de projetos, identificar tipos de arquivo, detectar linguagens de programacao e coletar metadados. Analisa: arquivos de configuracao, dependencias, estrutura de diretorios e padroes de codigo.

2. Deteccao de GPU (gpu_detector.py)

Detecta automaticamente a presenca de GPUs NVIDIA via CUDA. Consulta os drivers NVIDIA instalados e oferece opcao de escolha entre NVIDIA ou AMD. Esta informacao e crucial para direcionar as analises para o device correto.

3. Cliente LLM (llm_client.py)

Implementa comunicacao HTTP com LM Studio na porta 1234. Envia contexto da analise e recebe interpretacoes do modelo LLM. Trata erros de conexao e garante fallback gracioso se LM Studio nao estiver disponivel.

4. Persistencia (file_operations.py)

Salva todos os relatorios em formato JSON estruturado em diretorios separados (logs/nvidia/, logs/amd/, logs/system/). Implementa politicas de nomeacao consistente com timestamps.

6. Flask Web - Interface Remota

A versao Flask proporciona uma interface web moderna, responsiva e acessivel para usuarios que preferem uma experiencia grafica ao inves de linha de comando.

Arquitetura da Flask Web

```
app.py                (Aplicacao Flask principal)
├── templates/        (Templates HTML)
│   ├── index.html    (Pagina principal / Dashboard)
│   ├── cadastro.html (Formulario de cadastro de projetos)
│   ├── relatorios.html (Listagem e visualizacao de relatorios)
│   └── config.html    (Configuracoes da aplicacao)
```

— static/	(Arquivos estaticos)
— — css/	(Estilos CSS)
— — js/	(Scripts JavaScript)
— — images/	(Imagens e icones)
— routes/	(Rotas/endpoints da API)

Funcionalidades da Interface Web

Dashboard Principal

Mostra estatísticas gerais, projetos recentes, análises pendentes e últimos relatórios gerados. Proporciona uma visão holística do estado do sistema.

Cadastro de Projetos

Formulário web intuitivo para adicionar novos projetos. Permite upload de arquivos, seleção de tipo de projeto e configurações iniciais. Valida dados de entrada e fornece feedback em tempo real.

Gerenciamento de Relatórios

Lista todos os relatórios gerados, permite filtros por data/tipo/GPU, visualização detalhada de cada relatório com gráficos e métricas. Oferece opção de download em JSON ou PDF.

Configurações

Interface para configurar parâmetros de conexão com LM Studio, selecionar GPU preferida, definir políticas de retenção de logs e gerenciar credenciais de usuário.

7. DIFERENCIAL: IA Local com Aceleração GPU

Este é o grande diferencial do projeto ASPM IA FIAP! A integração nativa com modelos LLM locais via LM Studio, com suporte a aceleração por GPU NVIDIA CUDA e AMD ROCm, permite processamento de análises seguras sem dependência de APIs externas.

Fluxo Completo de Processamento com IA

PASSO 1: Usuário Seleciona IA Local

Usuário inicia uma análise e escolhe usar a opção de IA Local ao invés de APIs remotas. Na CLI, isto é selecionado no menu principal. Na Web, seleciona checkbox na configuração do projeto.

PASSO 2: Detecção Automática de GPU NVIDIA

Sistema verifica automaticamente disponibilidade de GPU NVIDIA via driver CUDA. Se detectada, NVIDIA é oferecida como opção preferencial por ser mais rápida. O módulo `gpu_detector.py` executa verificações de hardware.

PASSO 3: Menu de Seleção AMD ou NVIDIA

Sistema apresenta opções de GPU disponíveis. Usuário pode escolher NVIDIA CUDA, AMD ROCm ou CPU. Esta escolha afeta o device em que o modelo LLM será executado, influenciando velocidade e consumo de energia.

PASSO 4: Scanner Executa Análise Completa

Scanner lê recursivamente todo o projeto, coleta informações sobre arquivos, dependências, padrões de código e configurações. Gera um relatório estruturado contendo: tipos de arquivo, linguagens detectadas, tamanho total, estrutura de diretórios.

PASSO 5: Relatório JSON Salvo Separadamente

Resultado do scanner e salvo imediatamente em logs/nvidia/ ou logs/amd/ (dependendo da GPU selecionada). Arquivo segue padrao: analise_[timestamp]_[projeto].json. Facilita auditoria e rastreamento de todas as operacoes.

PASSO 6: CLI Comunica com LM Studio via HTTP

Sistema conecta ao LM Studio em localhost:1234. Envia o relatorio do scanner como contexto para o modelo LLM. Comunicacao e realizada via HTTP POST, permitindo transmissao de JSON estruturado.

PASSO 7: GPU Processa Modelo LLM

LM Studio carrega o modelo LLM selecionado (ex: Mistral, Llama 2, etc.) e executa inferencia no device escolhido (NVIDIA CUDA ou AMD ROCm). GPU acelera os calculos tensoriais, processando o modelo muito mais rapido que CPU. Resultado e retornado como resposta HTTP.

PASSO 8: Parecer da IA e Salvo Permanentemente

Resposta do modelo LLM (parecer sobre seguranca, vulnerabilidades, recomendacoes) e exibida ao usuario e salva permanentemente em arquivo JSON. Integrado ao relatorio anterior, criando documento completo e rastreavel.

Vantagens da Arquitetura com IA Local

- ✓ Privacidade Total: Dados nunca saem da rede local
- ✓ Sem Dependencias Externas: Funciona sem acesso a internet
- ✓ Velocidade: GPU NVIDIA/AMD fornece processamento ultrarapido
- ✓ Escalabilidade: Modelo roda localmente, sem limites de API
- ✓ Custo Zero: Sem taxas de API apos setup inicial

8. Stack Tecnológico

Tecnologias - API Python CLI

- Python 3.11+ : Linguagem principal
- FastAPI : Framework web de alta performance
- pydantic : Validacao de dados e schemas
- requests : Cliente HTTP para comunicacao com LM Studio
- pathlib : Manipulacao de caminhos de arquivo
- json : Serializacao de dados

Tecnologias - Flask Web

- Flask : Framework web para interface remota
- Jinja2 : Template engine HTML
- Bootstrap 5 : Framework CSS responsivo
- JavaScript/jQuery : Interatividade frontend
- SQLAlchemy : ORM para banco de dados (opcional)

IA e GPU

- LM Studio : Interface para carregar e executar modelos LLM
- NVIDIA CUDA : Aceleracao para GPUs NVIDIA
- AMD ROCm : Aceleracao para GPUs AMD Radeon
- Mistral / Llama 2 : Modelos LLM suportados

Containerizacao e Deployment

- Docker : Containerizacao de aplicacoes
- Docker Compose : Orquestracao de multiplos containers
- Dockerfile : Definicao de imagens customizadas

9. Guias de Execucao

Opcao 1: Execucao Local (Python CLI)

Passo 1: Clonar o repositório

```
git clone https://github.com/carmipa/CHALLENGE_2026_PRIDE_SECURITY.git
```

Passo 2: Acessar pasta da API Python

```
cd CHALLENGE_2026_PRIDE_SECURITY/api_python
```

Passo 3: Criar ambiente virtual Python

```
python -m venv .venv
```

Passo 4: Ativar ambiente virtual

```
# Windows:  
.venv\Scripts\activate  
# Linux/Mac:
```

```
source .venv/bin/activate
```

Passo 5: Instalar dependencias

```
pip install -r requirements.txt
```

Passo 6: Executar aplicacao

```
python main.py
```

Opcao 2: Execucao via Docker

Passo 1: Clonar repositório

```
git clone https://github.com/carmipa/CHALLENGE_2026_PRIDE_SECURITY.git
```

Passo 2: Entrar no diretório do projeto

```
cd CHALLENGE_2026_PRIDE_SECURITY
```

Passo 3: Executar Docker Compose

```
docker compose up --build
```

Passo 4: Acessar aplicacao

```
# CLI estara disponivel no terminal  
# Flask Web estara em http://localhost:5000
```

Opcao 3: Setup de IA Local com LM Studio

Passo 1: Instalar LM Studio

Visite <https://lmstudio.ai> e baixe o instalador para seu SO

Passo 2: Abrir LM Studio

Abra a aplicacao LM Studio. Navegue para a aba Models e faça download de um modelo (recomendado: Mistral 7B ou Llama 2)

Passo 3: Iniciar servidor LM Studio

Na aba Local Server, selecione o modelo desejado e clique Start Server. Server inicia em localhost:1234

Passo 4: Executar ASPM IA FIAP

```
python main.py (no terminal, com LM Studio rodando)
```

Passo 5: Selecionar opcao 4 - IA Local

No menu principal da CLI, escolha opcao 4 para usar IA Local com GPU

10. Equipe de Desenvolvimento

Turma 1TDCPV - FIAP 2026

Paulo Andre Carminati - RM570877

Gustav Quental Scorsi - RM569862

Andre Archanjo dos Santos Torres - RM570458

Luiz Carlos da Paixao dos Santos - RM573009

Victor Henrique de Barros Oliveira - RM570012

Repositorio GitHub

https://github.com/carmipa/CHALLENGE_2026_PRIDE_SECURITY

Contato Responsavel

Email: pabcarminati@gmail.com

Documento gerado em Maio de 2026